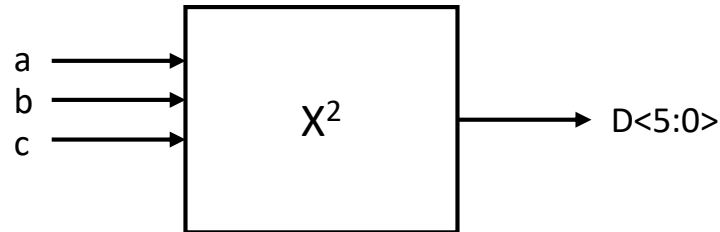


Esercizio 4: quadrato

Progetta un circuito che dato un numero in ingresso a 3 bit espresso in notazione complemento a 2, calcola il quadrato di tale numero in uscita.

Indispensabile comprendere il numero di bit in ingresso/uscita

- ingressi : 3bit
 - Spazio valori complemento a 2 è definito da $[-2^{N-1}, 2^{N-1}-1] \rightarrow$ range valori in ingresso $=[-4,3]$.
- Di conseguenza lo spazio d'uscita è definito nell'intervallo $[0,16]$
 - Per esprimere 16 in complemento a 2: 1 bit (segno) + 5 bit (16 max)



Esercizio 4: quadrato

a	b	c	Decimal in	Decimal out	D[5]	D[4]	D[3]	D[2]	D[1]	D[0]
0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	0	0	0	1
0	1	0	2	4	0	0	0	1	0	0
0	1	1	3	9	0	0	1	0	0	1
1	0	0	-4	16	0	1	0	0	0	0
1	0	1	-3	9	0	0	1	0	0	1
1	1	0	-2	4	0	0	0	1	0	0
1	1	1	-1	1	0	0	0	0	0	1

5 bit d'uscita → 5 espressioni logiche /mappe di Karnaugh

Possiamo analizzare meglio le colonne...

- $D[5] = D[1] = 0$ Non serve sintetizzare (i quadrati sono sempre ≥ 0)
- $D[4] = 1$ Solamente per la combinazione definita da $D[4]=ab'c'$
- $D[0] = c$ La sintesi circuitale è un filo!

Esercizio 4: quadrato

Risolviamo le mappe di Karnaugh per le due uscite:

a \ bc		D[3]			
		00	01	11	10
0	0	0	0	1	0
1	0	0	1	0	0

 $D[3] = a'bc + ab'c$ sintesi "1"

a \ bc		D[2]			
		00	01	11	10
0	0	0	0	0	1
1	0	0	0	0	1

 $D[2] = bc'$ sintesi "1"

Esercizio 4: quadrato

- Codice VHDL: Quadrato.vhd senza ieee.numeric_std.all

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

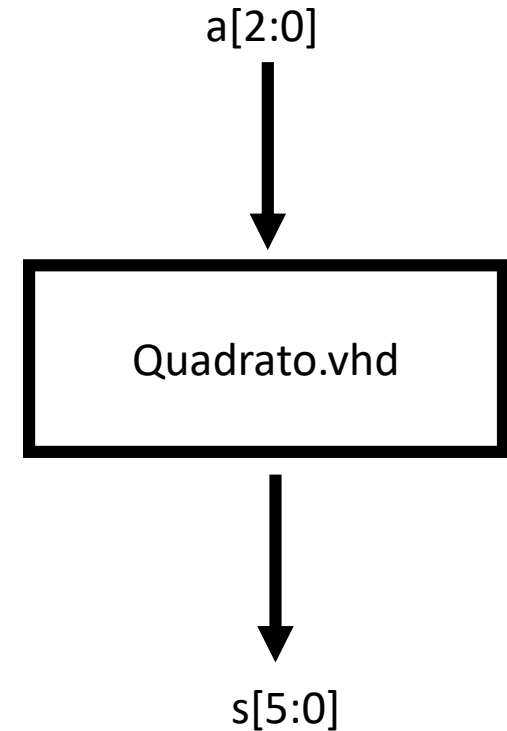
entity Quadrato is
Port (
    a:  in    std_logic_vector(2 downto 0);    -- a(2)=a, a(1)=b, a(0)=c
    s:  out   std_logic_vector(5 downto 0)      -- s[5:0]=D[5:0]
);
end Quadrato;

architecture Behavioral of Quadrato is

begin

    s(5) <= '0';
    s(4) <= (a(2) and not(a(1)) and not(a(0)));
    s(3) <= ((not(a(2)) and a(1) and a(0)) or (a(2) and not(a(1)) and a(0)));
    s(2) <= a(1) and not(a(0));
    s(1) <= '0';
    s(0) <= a(0);

end Behavioral;
```



Esercizio 4: quadrato

- Codice VHDL: Quadrato_testbench.vhd

```
entity Quadrato_testbench is
-- Port ( );
end Quadrato_testbench;

architecture Behavioral of Quadrato_testbench is

-- Dichiarazione componenti
component Quadrato
    Port (
        x: in std_logic_vector(2 downto 0);
        D: out std_logic_vector(5 downto 0)
    );
end component;

-- Dichiarazione segnali
signal x: std_logic_vector(2 downto 0) := (others => '0');
signal D: std_logic_vector(5 downto 0) := (others => '0');
```

```
begin

    uut:Quadrato
    port map(
        x=>x,
        D=>D
    );

    test_stim: process
    begin
        x<="000";
        wait for 10 ns;
        x<="001";
        wait for 10 ns;
        x<="010";
        wait for 10 ns;
        x<="011";
        wait for 10 ns;
        x<="100";
        wait for 10 ns;
        x<="101";
```

```
        wait for 10 ns;
        x<="110";
        wait for 10 ns;
        x<="111";

        wait;
        end process;

    end Behavioral;
```

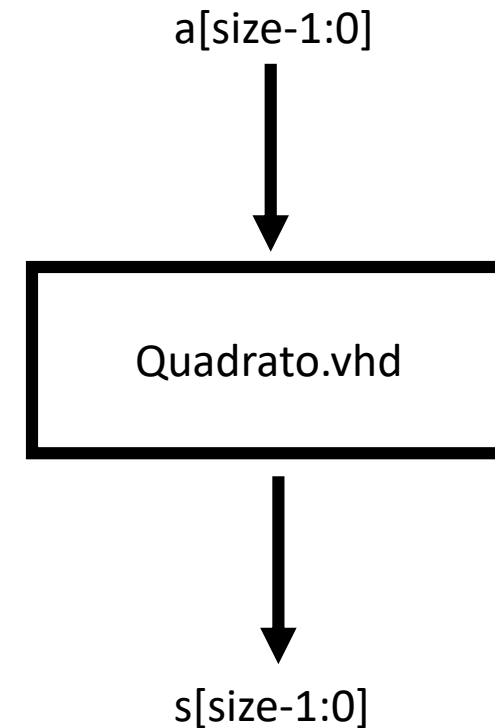
Esercizio 4: quadrato

- Codice VHDL: Quadrato.vhd con ieee.numeric_std.all

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use ieee.numeric_std.all;  
  
entity quadrato is  
generic (  
    size : integer := 8  
);  
port (  
    a : in signed( size - 1 downto 0 );  
    s : out signed( ( size * 2 ) - 1 downto 0 )  
);  
end entity;  
  
architecture behavioral of quadrato is  
begin  
    s <= a * a;  
end architecture;
```

Utile in fase di
istanziamento

utilizzo la libreria a
disposizione!



Esercizio 4: quadrato

- Codice VHDL: Quadrato_testbench.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity testbench is
end entity;

architecture behavioral of testbench is
    component quadrato is
        generic (
            size : integer := 8
        );
        port (
            a : in signed( size - 1 downto 0 );
            s : out signed( ( size * 2 ) - 1 downto 0 )
        );
    end component;
```

```
begin

-- Dichiarazione segnali
signal a : signed( 7 downto 0 );
signal s8 : signed( 15 downto 0 );
signal a4 : signed( 3 downto 0 );
signal s4 : signed( 7 downto 0 );
a4 <= a( 3 downto 0 );

-- Dichiarazione istanze
add32 : quadrato
port map (
    a => a,
    s => s8
);

add8 : quadrato
generic map ( size => 4 )
port map (
    a => a4,
    s => s4
);
```

Esercizio 4: quadrato

- Codice VHDL: Quadrato_testbench.vhd

```
process begin
  to_signed: restituisce il valore specificato in "value"
  in rappresentazione signed con un numero di bit pari a
  "size"
  a <= to_signed( 0, 8 );
  wait for 10 ns;
  a <= to_signed( 35, 8 );
  wait for 10 ns;
  a <= to_signed( 18, 8 );
  wait for 10 ns;

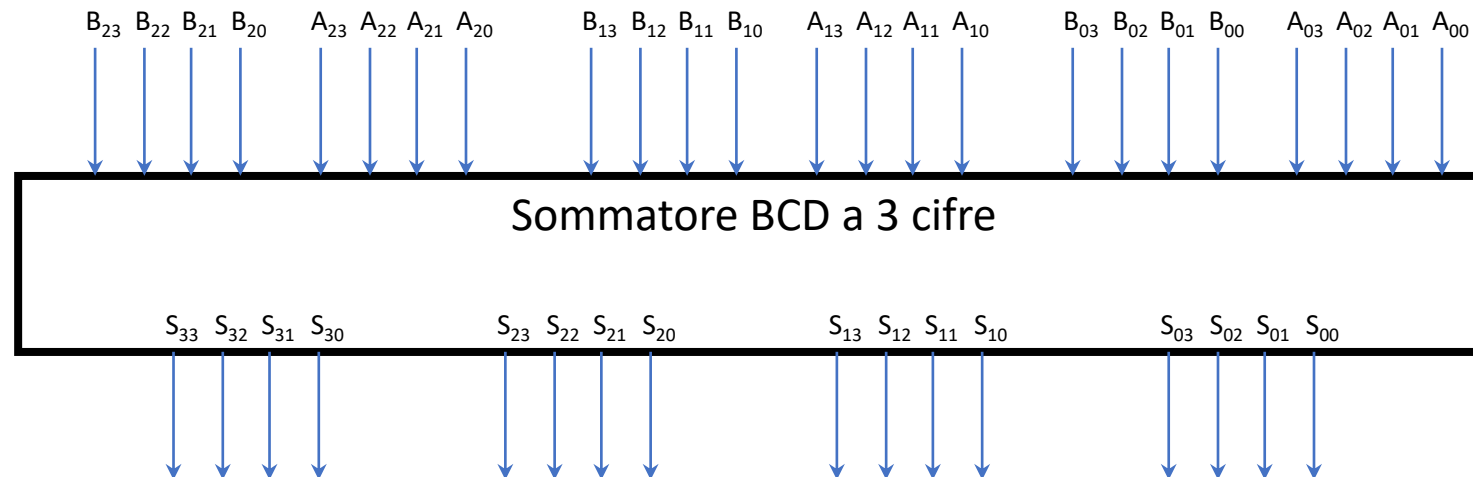
  a <= to_signed( -18, 8 );
  wait for 10 ns;
  a <= to_signed( 120, 8 );
  wait for 10 ns;
  a <= to_signed( 43, 8 );
  wait for 10 ns;
  a <= to_signed( -4, 8 );
  wait for 10 ns;
  a <= to_signed( -8, 8 );
  wait for 10 ns;
  a <= to_signed( 10, 8 );
```

```
wait for 10 ns;
  a <= to_signed( -20, 8 );
  wait for 10 ns;
  wait;
end process;

end architecture;
```

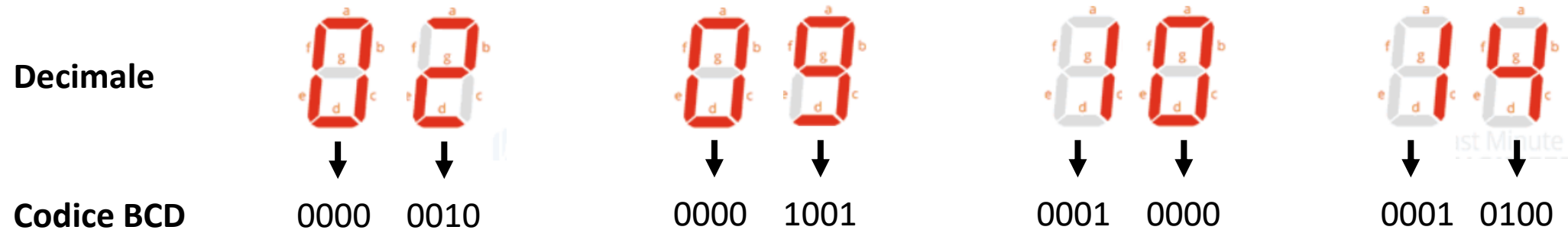

Esercizio 5: sommatore BCD

Progettare un sommatore per numeri a 3 cifre decimali codificati in BCD. Utilizzare per quanto possibile blocchi già disponibili, quali i sommatori binari a 4 bit, multiplexer, etc. Aggiungere il minimo di logica necessaria per gestire il carry correttamente. Scrivere il codice VHDL corrispondente e verificare il corretto funzionamento in simulazione.



Esercizio 5: sommatore BCD

La codifica BCD (Binary Coded Decimal) serve per rappresentare cifre decimali tramite il sistema binario. Ogni cifra decimale richiede quindi 4 bit (un *nibble*), in quanto bisogna codificare in binario numeri nel range $[0\div9]$. Vediamo alcuni esempi:



Si nota subito che parte delle combinazioni di bit non vengono sfruttate. In particolare le combinazioni mancanti per ogni nibble sono 6 (1010, 1011, 1100, 1101, 1110, 1111). Questo porta ad una perdita di efficienza rispetto al sistema di rappresentazione binario.

8 bit binario $\rightarrow 2^8=256$ valori nel range $[0,255]$

8 bit BCD $\rightarrow 10 \times 10=100$ valori nel range $[0,99]$

Esercizio 5: sommatore BCD

L'esercizio chiede di realizzare un sommatore BCD utilizzando un sommatore binario a 4 bit.

Problema → il sommatore binario in vari casi calcola un risultato non corretto

- I risultati **superiori a 9**, essendo codificati in binario, vengono sbagliati
- Il carry out si ha solo quando la somma delle due cifre BCD è superiore a 15

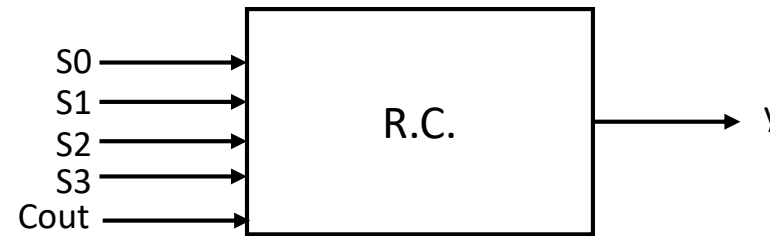


Soluzione → cerchiamo di “correggere” il risultato fornito dal sommatore binario quando viene sbagliato. Bisogna monitorare il risultato della somma attivando il riporto per tutti i casi tra 10 e 19.

- Le due cifre BCD arrivano al massimo a $9 + 9 = 18$. In aggiunta c'è l'eventuale Cin per arrivare a 19

Esercizio 5: sommatore BCD

La rete combinatoria che si occupa di capire quali casi correggere avrà quindi 5 bit di ingresso (4 bit di somma $S[3:0]$ più l'eventuale carry out), e un singolo bit di uscita (chiamiamolo "y").



Risolviamo tramite mappe di Karnaugh a variabili riportate: due mappe 4x4, una per la variabile riportata pari a 0 e l'altra per la variabile riportata pari a 1. Si compone poi il risultato finale tramite **espansione di Shannon**.

Come variabile riportata, scegliamo il carry out (Cout).

Esercizio 5: sommatore BCD

Uscita y per Cout = 0

Gli '1' sono per tutte le combinazioni S3,S2,S1,S0 che rappresentano un numero maggiore di 9

		S3 S1 \ S2 S0				
			00	01	11	10
00	0	0	1	0		
01	0	0	1	0		
11	0	0	1	1		
10	0	0	1	1		

Y0 = S3·S2 + S3·S1

Il risultato non è mai superiore a 19, quindi si mettono don't care per valori da 20 a 31

Uscita y=1 per Cout = 1

		S3 S1 \ S2 S0				
			00	01	11	10
00	1	-	-	-		
01	1	-	-	-		
11	1	-	-	-		
10	1	-	-	-		

Y1 =Cout

Una volta ottenuti i risultati parziali, si compone il risultato finale applicando il teorema di espansione di Shannon:

$$\begin{aligned}
 y &= C'_{out} \cdot y_0 + C_{out} \cdot y_1 = \\
 &= (S_3 S_2 + S_3 S_1) + C_{out}
 \end{aligned}$$

Esercizio 5: sommatore BCD

- Come correggere? → Notare però un errore sistematico!

Somma	Codice Binario Cout & S[3:0]	Riporto	Unità
9	01001	0	9
10	01010	1	0
11	01011	1	1
12	01100	1	2
19	10011	1	9

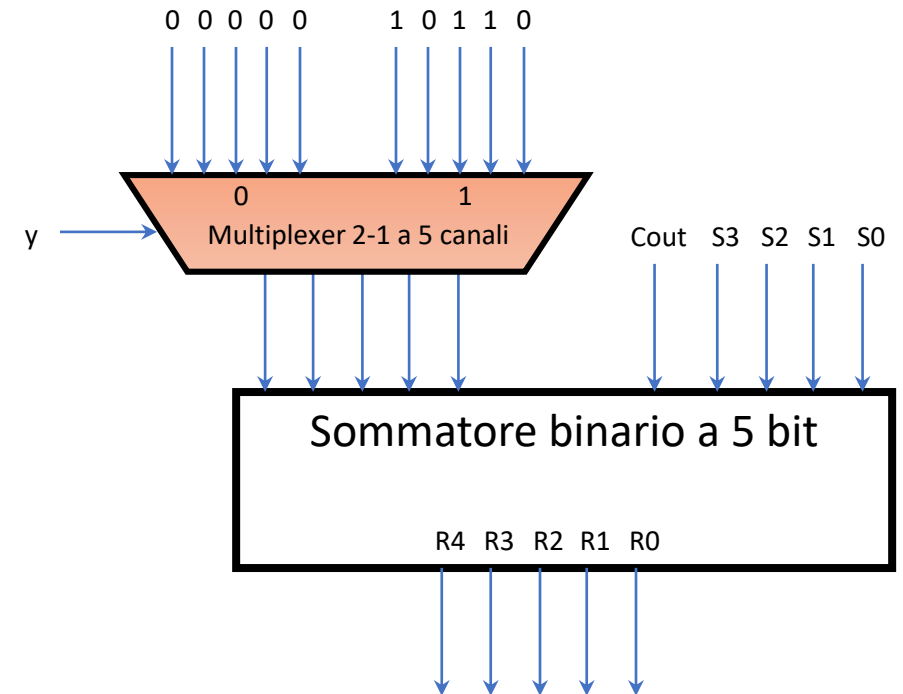


L unità sbaglia sempre
di 10 per numeri
maggiore di 9

- Basta sottrarre 10 al risultato della somma per ottenere il valore corretto di unità Ma non sempre: se il risultato è sotto 10 non c'è da fare nessuna correzione
 - La correzione è necessaria quando il risultato è maggiore di 9, cioè quando $y = 1$

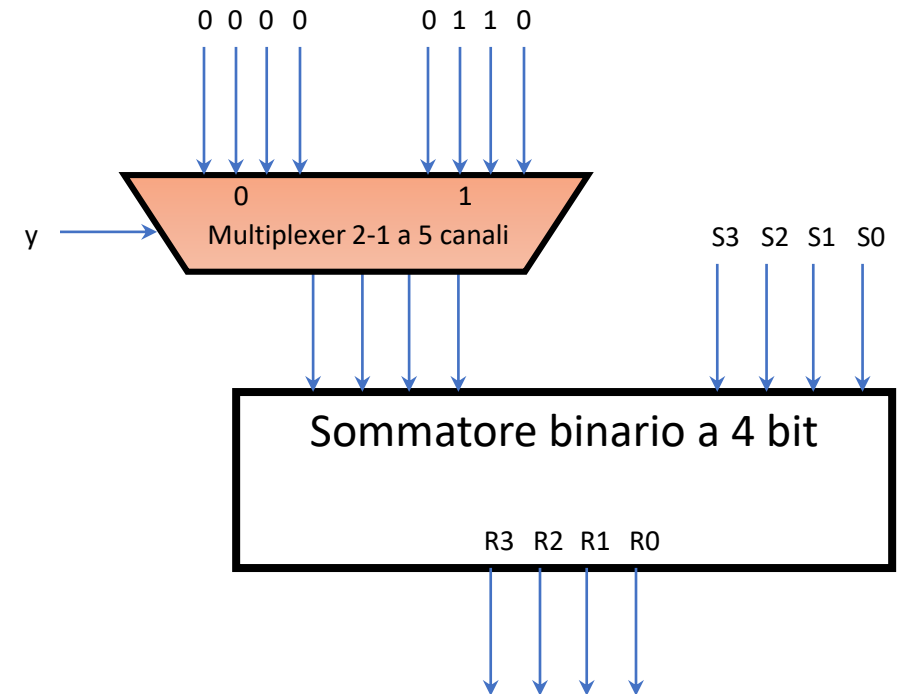
Esercizio 5: sommatore BCD

- Come sottraggo 10 (in base 10)?
 - Basta sommare il complemento a 2 di 10
 - In base 2 a 5 bit, 10 si esprime come 01010
 - Il complemento a 1 è 10101
 - Aggiungo ancora 1 per avere il complemento a 2: 10110
- Ma se il risultato era minore di 10 non dovevo aggiungere niente
 - In quel caso sommo 0: 00000



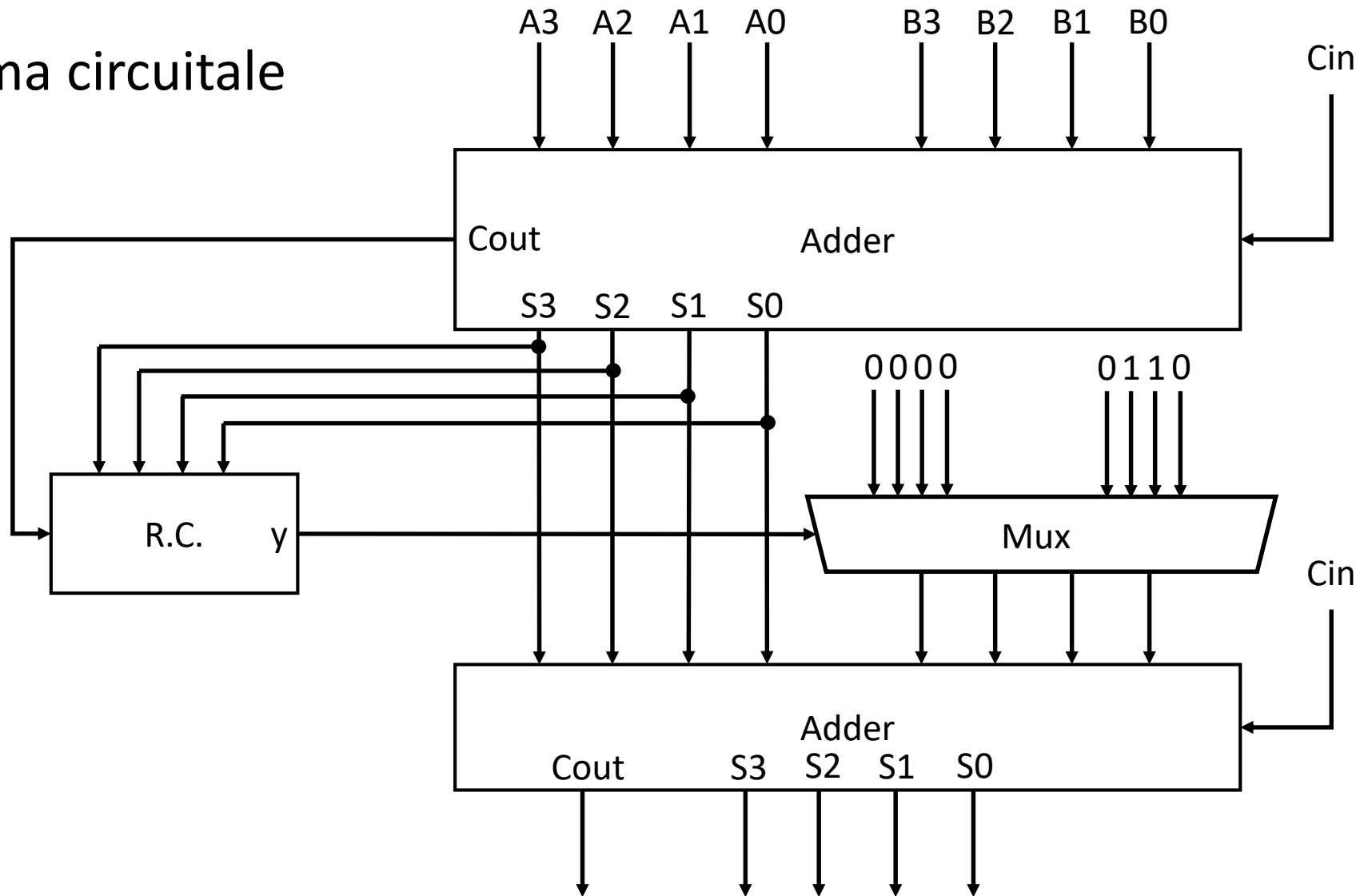
Esercizio 5: sommatore BCD

- Notare che il risultato R è sempre esprimibile in 4 bit
 - Se era superiore a 15 gli tolgo comunque 10, ed R è sempre tra 0 e 9
 - Quindi R4 sarà sempre a 0
 - Inoltre non si fa mai overflow, perché sottraggo 10 solo se il risultato era da 10 a 19
 - Quindi il quinto bit del sommatore non serve a niente
 - Posso usare un sommatore a 4 bit anche per la correzione



Esercizio 5: sommatore BCD

Schema circuitale

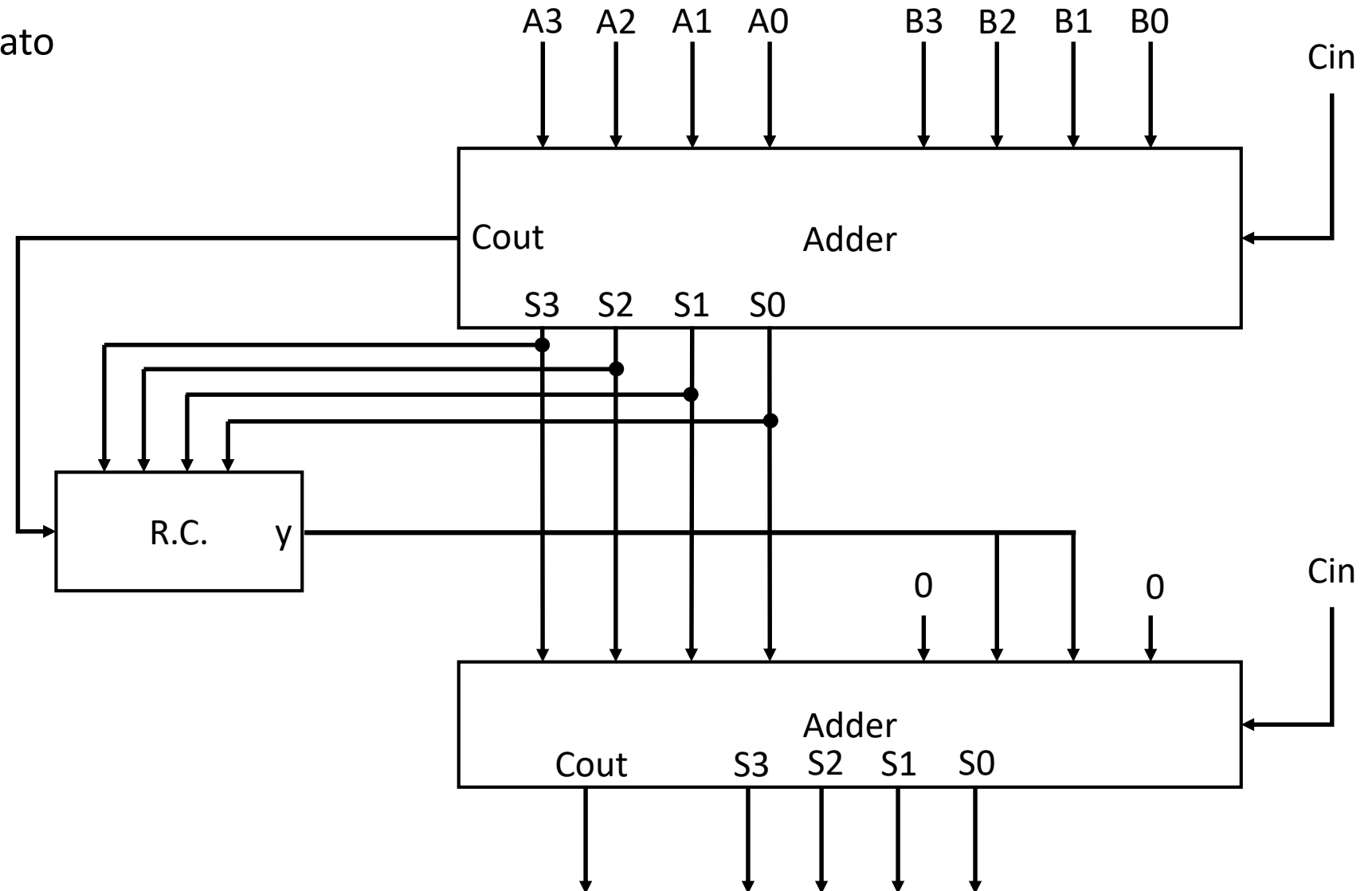


Esercizio 5: sommatore BCD

Schema circuitale semplificato

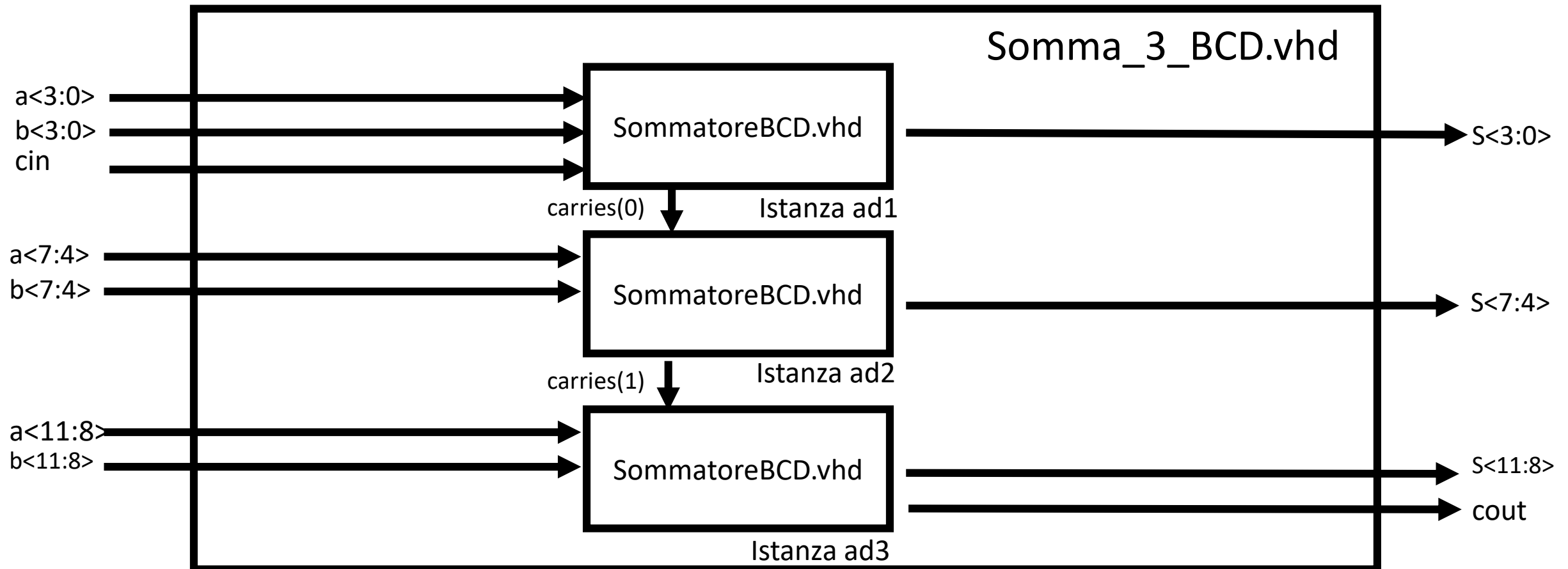
Il multiplexer ha ingressi a valori fissi

- L'uscita del multiplexer dipende solo da y
- Il primo e l'ultimo bit sono sempre a 0
- I due bit di mezzo sono a 1 se e solo se $y = 1$. Quindi sono uguali a carry_out!
- Per fare più cifre si mettono i sommatore in cascata!



Esercizio 5: sommatore BCD

- Codice VHDL: gerarchia



Esercizio 5: sommatore BCD

- Codice VHDL: Adder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity sommatore is
    generic (
        size : natural := 4
    );
    port (
        a, b : in unsigned( size - 1 downto 0 );
        cin : in std_logic;
        -- The output is one bit larger than the inputs and includes the carry out
        -- as the most significant bit
        s : out unsigned( size downto 0 )
    );
end sommatore;

architecture Behavioral of sommatore is

begin
    s <= ( "0" & a ) + ( "0" & b ) + ( "0000" & cin );
end Behavioral;
```

Esercizio 5: sommatore BCD

- Codice VHDL: SommatoreBCD.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity SommatoreBCD is
  port (
    a, b : in std_logic_vector( 3 downto 0 );
    cin : in std_logic;
    s : out std_logic_vector( 3 downto 0 );
    cout : out std_logic
  );
end SommatoreBCD;

architecture Behavioral of SommatoreBCD is

  -- internal signals declaration
  signal partial : unsigned( 4 downto 0 );
  signal correction : unsigned( 3 downto 0 );
  signal final : unsigned( 4 downto 0 );
  signal overflow : std_logic;

begin
```

Esercizio 5: sommatore BCD

- Codice VHDL: SommatoreBCD.vhd

```
begin

    -- Add the two inputs as if they were binary values, using a 4 bit adder
    add_partial : entity work.sommatore
        port map (
            a => unsigned( a ),
            b => unsigned( b ),
            cin => cin,
            s => partial
        );

    -- Verify if we have overflow
    -- partial <=9 --> overflow='0'
    -- partial > 9 --> overflow='1'

    overflow <= '0' when partial < to_unsigned( 10, 5 ) else '1';
    -- to_unsigned( 10, 5 ) converts 10 to 5bit unsigned vector
    cout <= overflow;
```

Esercizio 5: sommatore BCD

- Codice VHDL: SommatoreBCD.vhd

```
-- Choose the correction value
-- correction= 0  when overflow='0'
-- correction= 6  when overflow='1'

correction <= to_unsigned( 0, 4 ) when overflow = '0' else to_unsigned( 6, 4 );

-- Add the correction to the final result

add_final : entity work.sommatore
    port map (
        a => partial( 3 downto 0 ),
        b => correction,
        cin => '0',
        s => final
    );

-- The output is the 4 least significant bits
s <= std_logic_vector( final( 3 downto 0 ) );

end Behavioral;
```

Esercizio 5: sommatore BCD

- Codice VHDL: Somma_3_BCD.vhd

```
entity Somma_3_BCD is
    port (
        -- Three BCD inputs is 4 x 3 = 12 bits
        a : in std_logic_vector( 11 downto 0 );
        b : in std_logic_vector( 11 downto 0 );
        -- The output
        s : out std_logic_vector( 11 downto 0 );
        -- The carry out
        cout : out std_logic
    );
end Somma_3_BCD;

architecture Behavioral of Somma_3_BCD is

    signal carries : std_logic_vector( 1 downto 0 );

begin
    -- Instantiate the different adders
    ad1 : entity work.sommatoreBCD
        port map (
            a => a( 3 downto 0 ),
            b => b( 3 downto 0 ),
            cin => '0',
            s => s( 3 downto 0 ),
            cout => carries( 0 )
        );
```

```
    ad2 : entity work.sommatoreBCD
        port map (
            a => a( 7 downto 4 ),
            b => b( 7 downto 4 ),
            cin => carries( 0 ),
            s => s( 7 downto 4 ),
            cout => carries( 1 )
        );

    ad3 : entity work.sommatoreBCD
        port map (
            a => a( 11 downto 8 ),
            b => b( 11 downto 8 ),
            cin => carries( 1 ),
            s => s( 11 downto 8 ),
            cout => cout
        );

end Behavioral;
```


Esercizio 5: sommatore BCD

- Codice VHDL: testbench.vhd

```
entity testbench is
-- Port ( );
end testbench;

architecture Behavioral of testbench is

    signal a : std_logic_vector( 11 downto 0 );
    signal b : std_logic_vector( 11 downto 0 );
    signal s : std_logic_vector( 11 downto 0 );
    signal cout : std_logic;

begin

    dut : entity work.Somma_3_BCD
    port map (
        a => a,
        b => b,
        s => s,
        cout => cout
    );

end;
```

```
process
begin

    a <= "0000000000000";
    b <= "0000000000000";
    wait for 10 ns;
    a <= "0000000000001"; -- 1
    b <= "0000000000010"; -- 2
    wait for 10 ns;
    a <= "000000001000"; -- 8
    b <= "000000001001"; -- 9
    wait for 10 ns;
    a <= "000000101000"; -- 28
    b <= "000001001001"; -- 49
    wait for 10 ns;
    a <= "100110011001"; -- 99
    b <= "000000010000"; -- 10
    wait for 10 ns;
    wait;

end process;

end Behavioral;
```